# Hardware Authentication System

The design and implementation of a **Hardware Authentication System** using Verilog. The project's goal is to create a secure, reliable, and physical access control mechanism. Unlike software-based authentication, this system's logic is "burned" directly into a digital circuit, making it less vulnerable to common software attacks. The system validates a 4-digit password and triggers a physical alarm if multiple incorrect attempts are made.

## Functional Description

The system operates in three distinct phases, managed by the FSM logic within the always block.

**Reset Phase**: When the rst_n input is low, all internal registers (entered_pwd, digit_count, wrong_attempts) are immediately cleared to 0. The unlock and alarm outputs are also deactivated. This ensures a secure, predictable starting state.

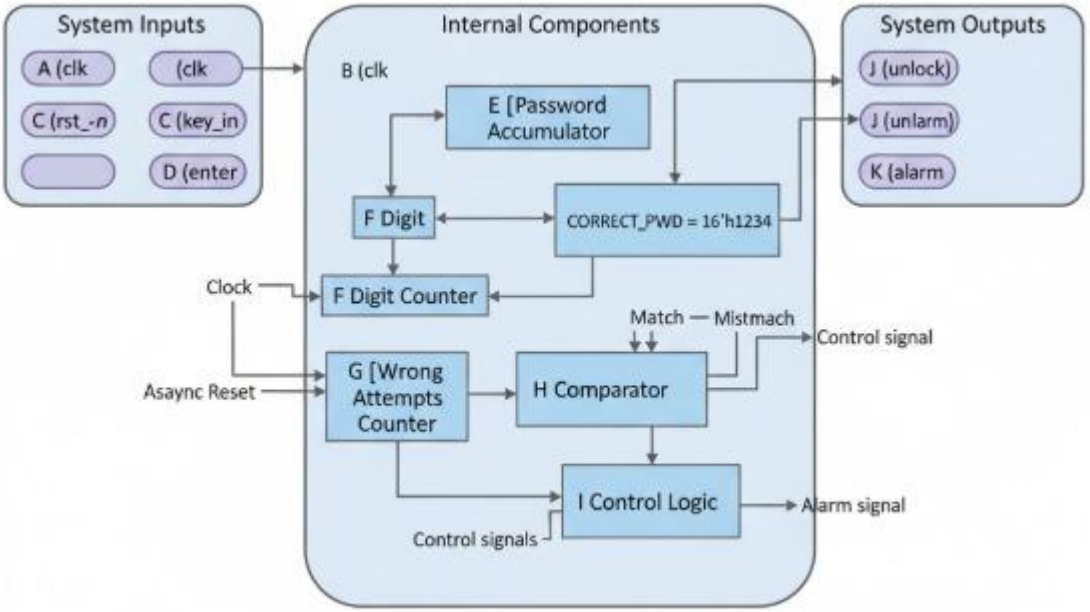
**Password Entry Phase**: When the enter button is not pressed, the system accumulates the password. With each rising clock edge, if digit_count is less than 4, the new key_in digit is concatenated to entered_pwd, and digit_count is incremented.

**Verification Phase**: This phase is triggered by the enter signal. The circuit performs a direct comparison between the entered_pwd and CORRECT_PWD.

**Success**: If the passwords match, unlock is set to 1, and wrong_attempts is reset to 0.

**Failure**: If the passwords do not match, unlock remains 0. The wrong_attempts counter is incremented. If the count reaches 3, the alarm signal is activated.

# BLOCK DIAGRAM:




# CODE:

```
module password_checker(
    input clk,
    input rst_n,
    input [3:0] key_in,    // 4-bit digit input
    input enter,           // Enter button
    output reg unlock,     // Unlock signal
    output reg alarm       // Alarm signal
);

    // Stored correct password = 1234 (BCD digits: 0001 0010 0011 0100)
    parameter CORRECT_PWD = 16'h1234;

    reg [15:0] entered_pwd;
    reg [1:0] digit_count;
    reg [1:0] wrong_attempts;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            entered_pwd    <= 16'b0;
            digit_count    <= 0;
            unlock         <= 0;
            alarm          <= 0;
            wrong_attempts<= 0;
        end
        else if (enter) begin
            if (entered_pwd == CORRECT_PWD) begin
                unlock         <= 1;
                alarm          <= 0;
```

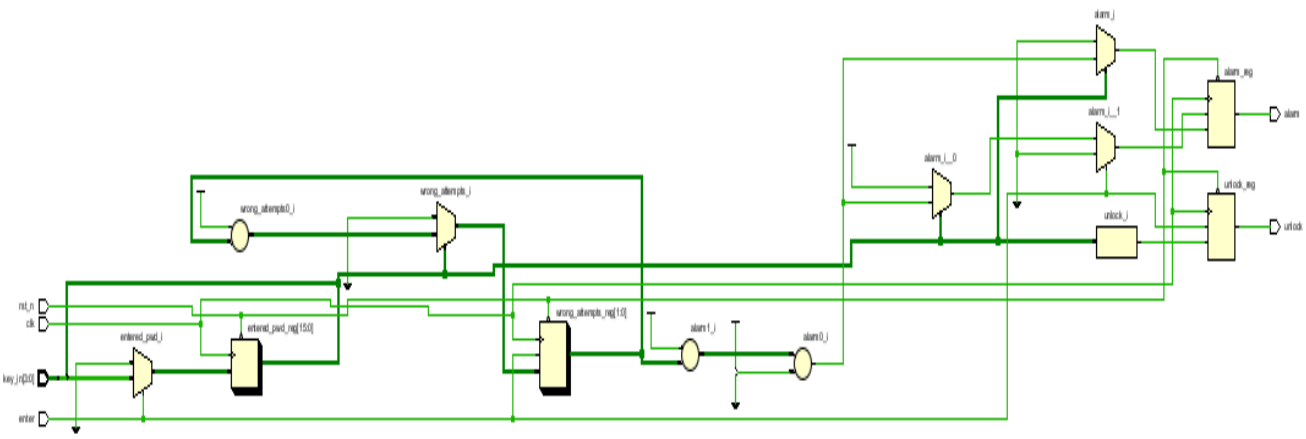
```
                wrong_attempts<= 0;
            end
            else begin
                unlock <= 0;
                wrong_attempts <= wrong_attempts + 1;
                if (wrong_attempts + 1 == 3)  // alarm on 3rd wrong attempt
                    alarm <= 1;
            end
            entered_pwd <= 16'b0;
            digit_count <= 0;
        end
        else begin
            if (digit_count < 4) begin
                entered_pwd <= {entered_pwd[11:0], key_in};
                digit_count <= digit_count + 1;
            end
        end
    end
endmodule
```

# Test Bench:

```
module password_tb;

    reg clk, rst_n, enter;
    reg [3:0] key_in;
    wire unlock, alarm;

    // Instantiate the DUT
    password_checker uut (
        .clk(clk),
        .rst_n(rst_n),
        .key_in(key_in),
        .enter(enter),
        .unlock(unlock),
        .alarm(alarm)
    );

    // Clock generation
    always #5 clk = ~clk;

    // Task: Enter a 4-digit password
    task enter_password(input [15:0] pwd);
        integer i;
        reg [3:0] digit;
        begin
            for (i=0; i<=12; i=i+4) begin
                digit = pwd[15-i -: 4];  // Extract 4 bits from MSB side
                @(posedge clk);
```

```
            key_in = digit;
        end
        @(posedge clk);
        enter = 1;  // Press enter
        @(posedge clk);
        enter = 0;
    end
endtask

initial begin
    // Monitor outputs
    $monitor("Time=%0t | entered=%h | unlock=%b | alarm=%b | wrong_attempts=%0d",
             $time, uut.entered_pwd, unlock, alarm, uut.wrong_attempts);

    // Initialize
    clk = 0;
    rst_n = 0;
    key_in = 0;
    enter = 0;

    // Reset
    #20 rst_n = 1;


     // Test 1: Correct password (1234)
     $display("\nTest 1: Correct password (Expect unlock=1, alarm=0)");
     enter_password(16'h1234);
     #20;

     // Test 2: Wrong password (5678)
     $display("\nTest 2: Wrong password (Expect unlock=0, alarm=0)");
     enter_password(16'h5678);
     #20;

     // Test 3: Wrong password (9999)
     $display("\nTest 3: Wrong password (Expect unlock=0, alarm=0)");
     enter_password(16'h9999);
     #20;

     // Test 4: Wrong password again -> Alarm should trigger
     $display("\nTest 4: Wrong password (Expect unlock=0, alarm=1)");
     enter_password(16'h1111);
     #20;

     #100 $stop;
    end
endmodule
```

## Schematic Diagram:



## Utilization Report & Power Analysis:

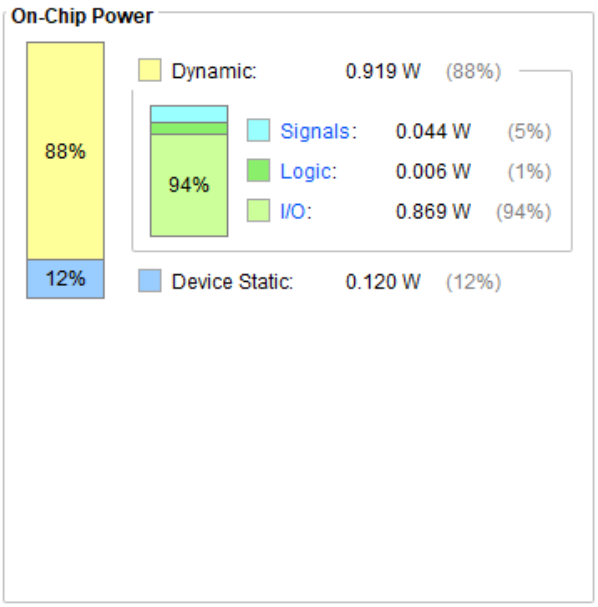
xc7z020clg484-1

| Name | Slice LUTs (53200) | Slice (13300) | LUT as Logic (53200) | Bonded IOB (200) |
|---|---|---|---|---|
| N comparator_case | 2 | 1 | 2 | 5 |

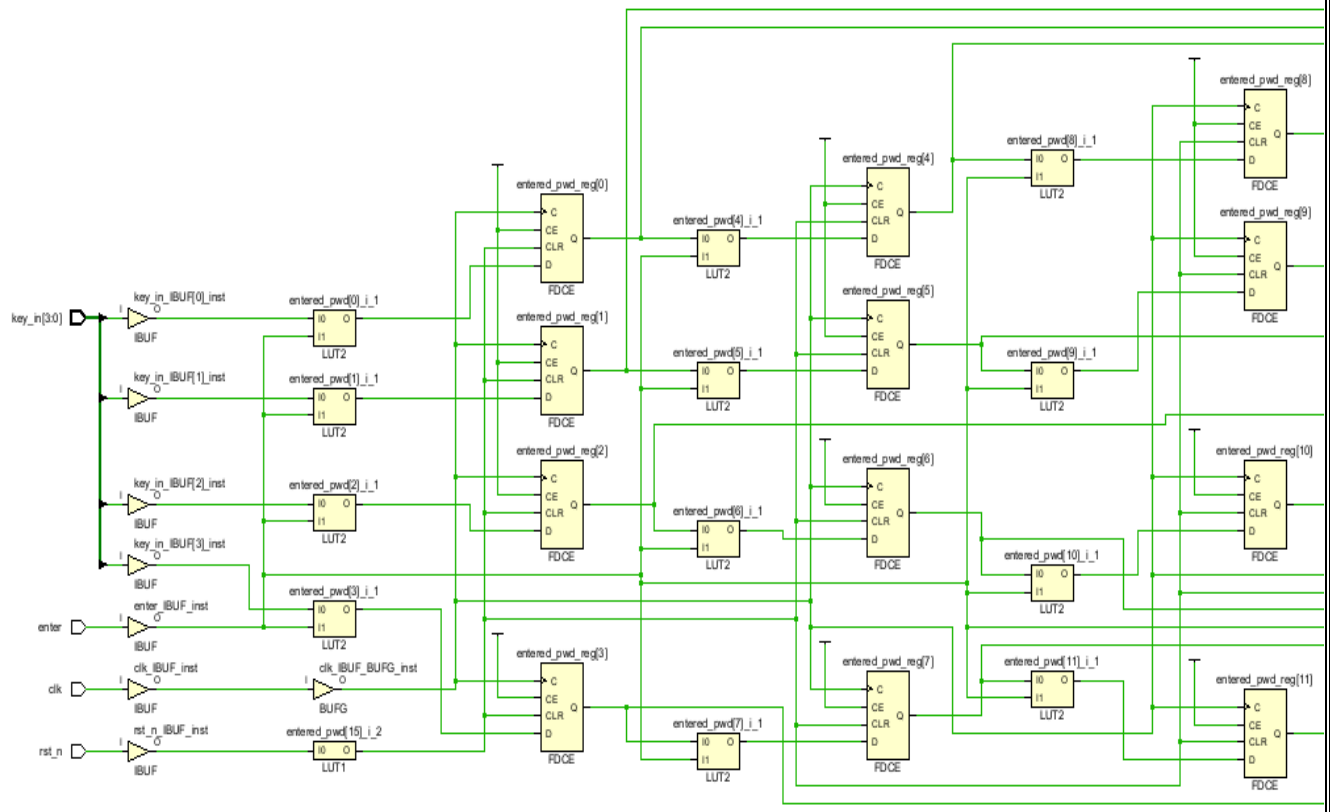
Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

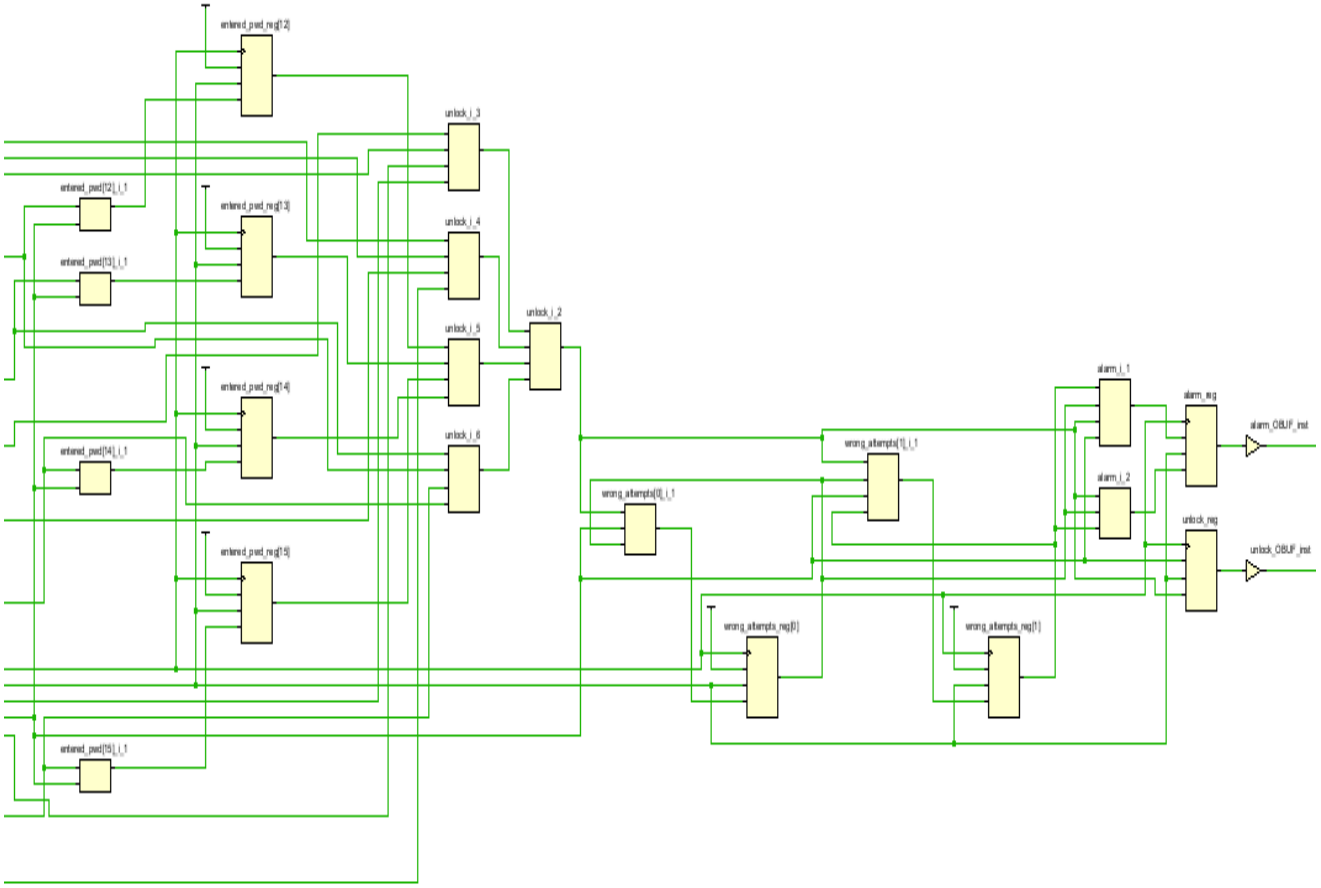
| | |
|---|---|
| **Total On-Chip Power:** | **1.039 W** |
| **Design Power Budget:** | **Not Specified** |
| **Process:** | typical |
| **Power Budget Margin:** | **N/A** |
| **Junction Temperature:** | **37.0°C** |
| Thermal Margin: | 48.0°C (4.0 W) |
| Ambient Temperature: | 25.0 °C |
| Effective ϑJA: | 11.5°C/W |
| Power supplied to off-chip devices: | 0 W |
| Confidence level: | Low |

Launch Power Constraint Advisor to find and fix invalid switching activity



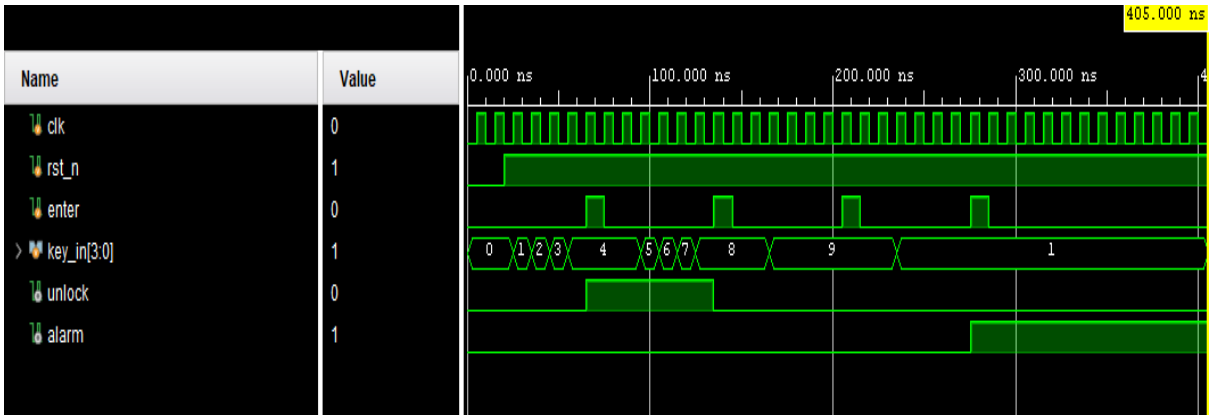

## Implementation Schematic Diagram:


key_in[3:0]
key_in_IBUF[0]_inst
key_in_IBUF[1]_inst
key_in_IBUF[2]_inst
key_in_IBUF[3]_inst
enter
enter_IBUF_inst
clk
clk_IBUF_inst
clk_IBUF_BUFG_inst
rst_n
rst_n_IBUF_inst
IBUF
BUFG
entered_pwd[0]_i_1
entered_pwd[1]_i_1
entered_pwd[2]_i_1
entered_pwd[3]_i_1
entered_pwd[15]_i_2
LUT2
LUT1
entered_pwd_reg[0]
entered_pwd_reg[1]
entered_pwd_reg[2]
entered_pwd_reg[3]
entered_pwd_reg[4]
entered_pwd_reg[5]
entered_pwd_reg[6]
entered_pwd_reg[7]
entered_pwd_reg[8]
entered_pwd_reg[9]
entered_pwd_reg[10]
entered_pwd_reg[11]
FDCE


entered_pwd_reg[12]
entered_pwd_reg[13]
entered_pwd_reg[14]
entered_pwd_reg[15]
unlock_i_3
unlock_i_4
unlock_i_5
unlock_i_6
unlock_i_2
wrong_attempts[0]_i_1
wrong_attempts[1]_i_1
wrong_attempts_reg[0]
wrong_attempts_reg[1]
alarm_i_1
alarm_i_2
alarm_reg
unlock_reg
alarm_OBUF_inst
unlock_OBUF_inst

**Power Analysis:**

**Simulation Results:**




**Conclusion:**

The developed hardware authentication system successfully demonstrates a robust and reliable security mechanism. By implementing the authentication logic in hardware, the system is less susceptible to common software vulnerabilities. The design efficiently manages password accumulation, verification, and includes a critical alarm mechanism for handling security breaches. This project provides a foundational understanding of how digital circuits can be designed and implemented for secure, real-world applications.